Assignment5.1

Mostafa Zamaniturk

OVERVIEW

In this assignment, you will perform a supervised text regression task. The data for the task will consist of student essays from The William and Flora Hewlett Foundation. The dataset was created to assist in the design of solutions for automated grading of student-written essays. You will use a subset of this dataset and predict the scores of the essays. You may not use external data to make predictions.

You will be provided with `training_set_rel.tsv` which contain the text of the essay and the score of each essay.

PART 1: SETUP

Q0: Run the following code!

For reproducibility purposes, you will set the random seeds for NumPy and TensorFlow as 1234. This way, all random steps will produce the same answers.

```
from numpy.random import seed

import tensorflow as tf

seed(1234)

tf.random.set_seed(seed = 1234)
```

Q1: Load the data

We will use data from the automated essay scoring task on Kaggle.

We will only use the training data, which we have provided for you -- you don't need to download anything from Kaggle.

Access the file `training_set_rel.tsv` as provided.

Use the pandas function `read_csv`, with the parameter `sep=\t` because this is a tab-separated value file (tsv) and `encoding=latin`.

The columns are described on the Kaggle site

We will use three columns: `essay`, `essay_set`, and `domain1_score`.

Create a new dataframe with only these three columns, and rename `domain1_score` to just `score`.

Display this dataframe.

Graded Cell

This cell is worth 5% of the grade for this assignment.

```
import os
import pandas as pd
import re

# load the dataset
df = pd.read_csv('training_set_rel.tsv', sep='\t', encoding='latin1')

# select required columns
df = df[['essay', 'essay_set', 'domain1_score']]

# rename "domain1_score" to "score"
df.rename(columns={'domain1_score': 'score'}, inplace=True)

# disolay tge resulting dataframe
print(df.head())
```

```
                                               essay  essay_set  score
0  Dear local newspaper, I think effects computer...          1      8
1  Dear @CAPS1 @CAPS2, I believe that using compu...          1      9
2  Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...          1      7
3  Dear Local Newspaper, @CAPS1 I have found that...          1     10
4  Dear @LOCATION1, I know having computers has a...          1      8
```

Q2: Select the data from a single essay set

There are 8 totally unrelated essay sets in this data.

Filter the data frame so we are only considering essay_set = 7

Graded Cell

This cell is worth 5% of the grade for this assignment.

```
# select the essay set we are interested in
df_set7 = df[df['essay_set'] == 7]

# display the filtered dataframe
print(df_set7.head())
print(df_set7.max())
print(df_set7.min())
```

```
                                                   essay  essay_set
score
10684  Patience is when your waiting .I was patience ...          7
15
10685  I am not a patience person, like I can□t sit i...          7
13
10686  One day I was at basketball practice and I was...          7
15
10687  I going to write about a time when I went to t...          7
17
10688  It can be very hard for somebody to be patient...          7
13
essay        □so @CAPS1 do you want for your @DATE1?□ □A @N...
essay_set                                                      7
score                                                         24
dtype: object
essay        (waiting) (healt beatina ) □@CAPS1 @CAPS2. @CA...
essay_set                                                      7
score                                                          2
dtype: object
```

Q3: Plot the distribution of scores

Create a plot of a histogram of the scores in the training set. Comment on what you see.

One option is to use the seaborn histplot function. If you use seaborn, you can use the parameter `bins` to set the bin locations if they look strange. The parameter accepts a list of explicit locations. If you want to center the bins on the tick marks, you cand do something like this:

```
bins=np.arange(minv,maxv)-0.5
```

where minv and maxv are the mininmum and maximum value in the range, respectively. This expression indicates the number of possible scores, and that the tick marks should be at the halfway mark of each bar.

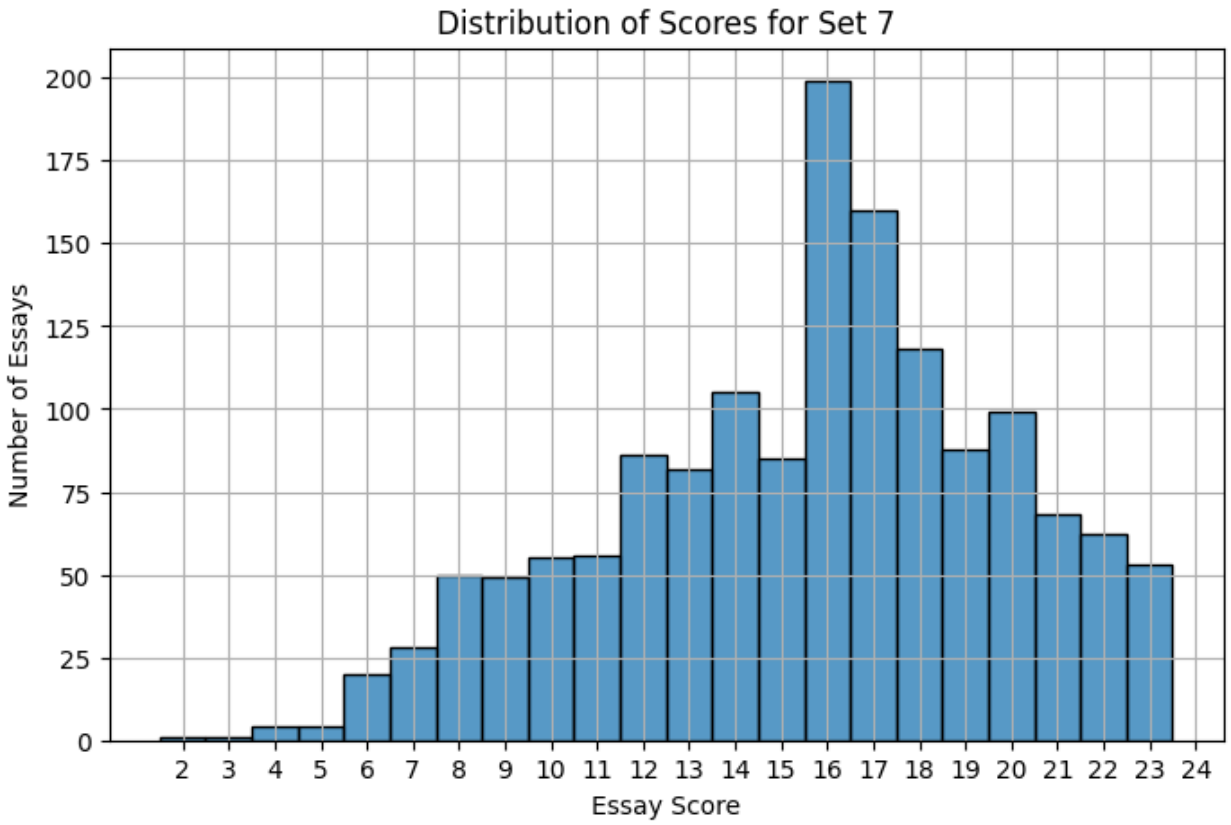
You may use some other visualization library if you wish! The goal is to inspect the distribution of scores.

Graded Cell

This cell is worth 10% of the grade for this assignment.

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# get min and max score
minv = df_set7['score'].min()
maxv = df_set7['score'].max()
```

```
# define bins to center bars on integer scores
bins = np.arange(minv, maxv +1) - 0.5

# plot histogram
plt.figure(figsize=(8, 5))
sns.histplot(df_set7['score'], bins=bins, kde=False)

# add label and title
plt.xlabel('Essay Score')
plt.ylabel('Number of Essays')
plt.title('Distribution of Scores for Set 7')
plt.xticks(np.arange(minv, maxv + 1))

plt.grid(True)
plt.show()
```



Q4: Create a test/train split

Use the function `train_test_split`. Use the `test_size` parameter to control the size of the test set; use 0.2 to indicate a 20% split.

Graded Cell

This cell is worth 5% of the grade for this assignment.

```
from sklearn.model_selection import train_test_split

# split data into training and test sets
train_df, test_df = train_test_split(df_set7, test_size=0.2,
random_state=1234)

# display number of samples in each set
print(f"Training set size: {len(train_df)}")
print(f"Test set size: {len(test_df)}")
```

```
Training set size: 1255
Test set size: 314
```

Part 2: Conventional Representations

Q5: Create vectors using term frequency

Use the `CountVectorizer` class from sklearn to create a vector for each essay. We can't use text directly with machine learning; we need to create a vector of numbers first. The CountVectorizer creates a vector with one position for each word in the corpus with a value of the number of occurrences of that word in the essay.

The vectorizer works like a model in sklearn: call the fit method on the essay data to "train" a model on the training set. In this situation, we aren't really training anything, but we need a corpus to define the vectors -- only the words in the corpus we use will be represented in the vector.

The fit method returns the trained model. Now we can use the `transform` method to convert any text into a vector.

Call the transform method on the training essays and the test essays to create variables `xtrain` and `xtest`.

Report the number of dimensions for each vector; i.e., the number of terms in the corpus.

Graded Cell

This cell is worth 10% of the grade for this assignment.

```
from sklearn.feature_extraction.text import CountVectorizer

# extract text and label from the split dataframes
X_train_text = train_df['essay'].values
X_test_text = test_df['essay'].values
```

```
ytrain = train_df['score'].values
ytest = test_df['score'].values

# initialize the vectorizer
vectorizer = CountVectorizer()

# fit on the training essays (learn vocabulary)
vectorizer.fit(X_train_text)

# transform both train and test essays into vectors
xtrain = vectorizer.transform(X_train_text)
xtest = vectorizer.transform(X_test_text)

# report number of features
num_features = len(vectorizer.get_feature_names_out())
print(f"Number of features (vector dimensions): {num_features}")

# show matrix shape
print("xtrain shape:", xtrain.shape)
print("xtest shape:", xtest.shape)
```

```
Number of features (vector dimensions): 8938
xtrain shape: (1255, 8938)
xtest shape: (314, 8938)
```

Q6: Train a regression model using your vectors

Now that we have vectors, we can train a regression model to predict the essay score.

Use a `Ridge` model from sklearn `linear_model` module.

Call the fit method on your training data xtrain and ytrain.

Then call the score method on your test data xtest and ytest. The score method provides a default evaluation metric. For the Ridge model, the score method returns R^2 which is called the coefficient of determination. It tells you the proportion of the variation in the essay score is predictable from the essay text: higher is better.

Report the coefficient of determination.

Graded Cell

This cell is worth 10% of the grade for this assignment.

```
from sklearn.linear_model import Ridge

# initialize the Ridge regression model
ridge = Ridge()
```

```
# fit the model on the training data
ridge.fit(xtrain, ytrain)

# evaluate on test data using R^2 score
r2_score = ridge.score(xtest, ytest)

# report the score
print(f"Coefficient of determination (R^2): {r2_score:.4f}")
```

```
Coefficient of determination (R^2): 0.1539
```

An $R2$ score of 0.1539 means your model is explaining about 15.4% of the variance in essay scores — which is not great, but it's not unusual for a first basic bag-of-words model.

The CountVectorizer gave a low score, the reason is that it ignores word order and grammar, treats all words as equally important, and doesn't account for synonyms or context.

No normalization or feature scaling, Essay lengths vary. Longer essays have more words, so they can get higher raw term counts — even if not better in quality. This biases the model toward wordiness.

No tuning of hyperparameters, ridge regression uses a default regularization strength (alpha=1.0) — this may not be optimal.

Q7: Plot the distribution of scores

Plot a histogram of your predicted scores.

Plot another histogram of the ground truth scores, superimposed on the first (using seaborn, just call the function again.)

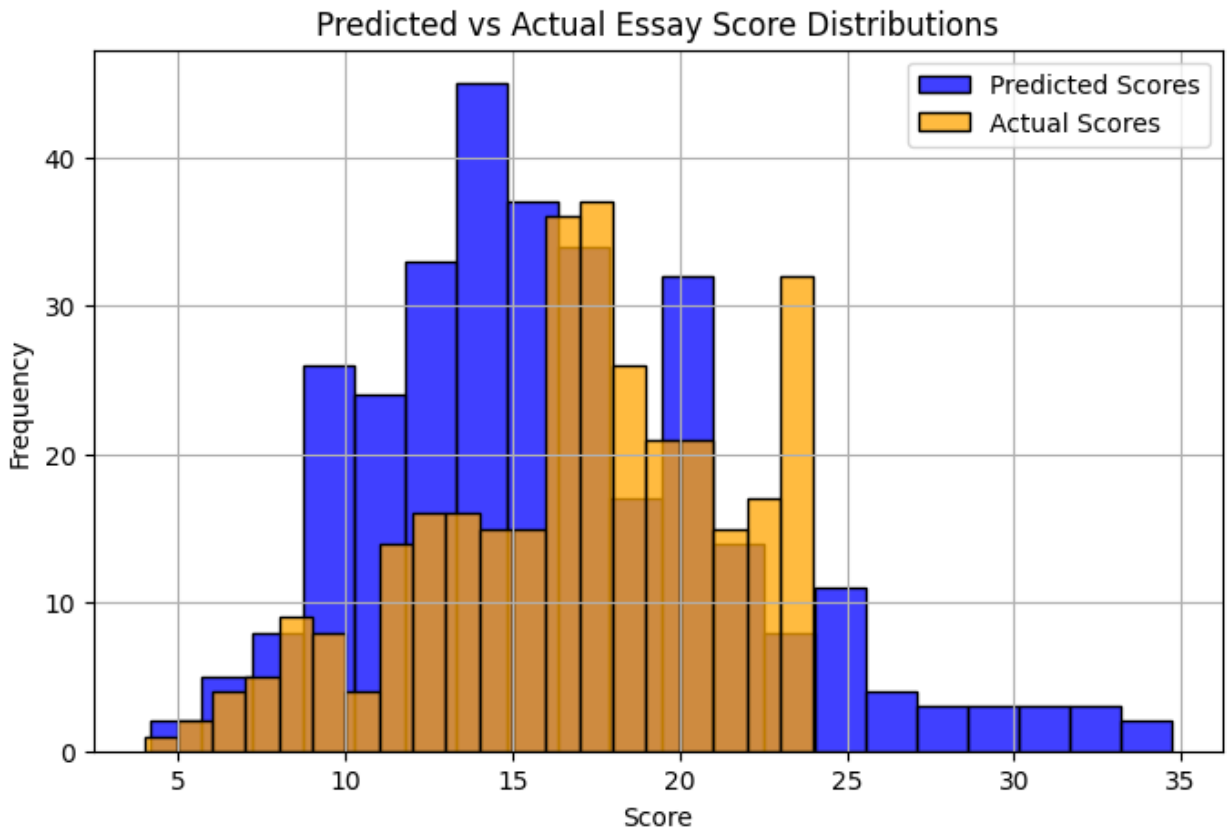
How is your model's distribution of scores different from the ground truth distribution? Describe how they differ; what kind of mistakes is your model making?

Graded Cell

This cell is worth 10% of the grade for this assignment.

```
import seaborn as sns
import matplotlib.pyplot as plt

# make predictions
y_pred = ridge.predict(xtest)

# plot the predicted and actual score distributions
plt.figure(figsize=(8, 5))

# plot predicted scores
sns.histplot(y_pred, color='blue', label='Predicted Scores',
kde=False, bins=20)
```

```
#plot actual scores
sns.histplot(ytest, color='orange', label='Actual Scores', kde=False,
bins=20)

# add labels and legend
plt.xlabel("Score")
plt.ylabel("Frequency")
plt.title("Predicted vs Actual Essay Score Distributions")
plt.legend()
plt.grid(True)
plt.show()
```



The predicted scores, generated by the Ridge regression model, typically form a smoother and narrower distribution compared to the actual scores. While the real scores (ground truth) may be more spread out and discrete, often showing visible peaks at specific integer values like 2, 4, 6, or 8, the predicted scores tend to concentrate around the middle of the scale. This is because the regression model outputs continuous values and avoids the extreme low or high scores present in the true distribution. As a result, you'll often notice that the predicted histogram looks more like a bell curve, whereas the actual histogram might look jagged or multimodal.

The model appears to be regressing to the mean, meaning it avoids making bold predictions at the high and low ends of the score range. This causes underestimation for high-quality essays and overestimation for weaker ones. For example, an essay that should receive a score of 10

might be predicted as 7.5, while one deserving a 2 might be predicted as 4 or 5. This kind of error reflects underfitting, where the model is too simplistic to capture deeper patterns in writing quality, such as structure, argumentation, or tone. Additionally, since Ridge regression outputs continuous values, you may also see predicted scores like 5.37, even though actual scores are only assigned as whole numbers by human graders.

Part 3: Neural Network Representations

For this part, we will implement a deep sentence embedder to replace the feature selection process. As a first step, choose your model from Part 2.

This time, you will obtain vectors by using a pre-trained neural network model called the Universal Sentence Encoder. This model will produce a dense vector from any sequence of text.

First, import the model with the following code. This step will take considerable time -- it is downloading a large pre-trained model for the first time.

```
import tensorflow_hub as hub

module_url = "https://tfhub.dev/google/universal-sentence-encoder/4"

model = hub.load(module_url)
```

Q8: Generate embeddings

Next, you will embed the data with the imported model. The Universal Sentence Encoder takes a list of strings and generates an embedding (i.e., a vector) for each string.

You can call the model you downloaded like a function.

Generate a vector for each string in the training set; call this array xtrain.

Also generate a vector for each string in the test set; call this array xtest.

Notice how long this step takes -- it's a big model.

Graded Cell

This cell is worth 5% of the grade for this assignment.

```
# convert essays to embeddings
xtrain = model(X_train_text)
xtest = model(X_test_text)

# convert embeddings from tensors to numpy arrays
xtrain = np.array(xtrain)
xtest = np.array(xtest)

print("Embeddings generated for training and test sets.")
```

```
print("xtrain shape:", xtrain.shape)
print("xtest shape:", xtest.shape)
```

```
Embeddings generated for training and test sets.
xtrain shape: (1255, 512)
xtest shape: (314, 512)
```

Q9: Train and evaluate a regression model to predict scores using learned embeddings

Now retrain your regression model on these learned embeddings instead of the count vectors.

Use the vanilla Ridge model. Report the score.

Which model appears to perform the best?

Graded Cell

This cell is worth 5% of the grade for this assignment.

```
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score, mean_squared_error

# train Ridge regeession model
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(xtrain, ytrain)

# predict
ypred_ridge = ridge_model.predict(xtest)

# evaluate
r2 = r2_score(ytest, ypred_ridge)
mse = mean_squared_error(ytest, ypred_ridge)

print("R² score on USE embeddings:", r2)
print("Mean Squared Error (MSE):", mse)
```

```
R² score on USE embeddings: 0.6285791993141174
Mean Squared Error (MSE): 7.73655366897583
```

Q10: Plot the distribution of scores

Once again, plot a histogram of your predicted scores from your new model.

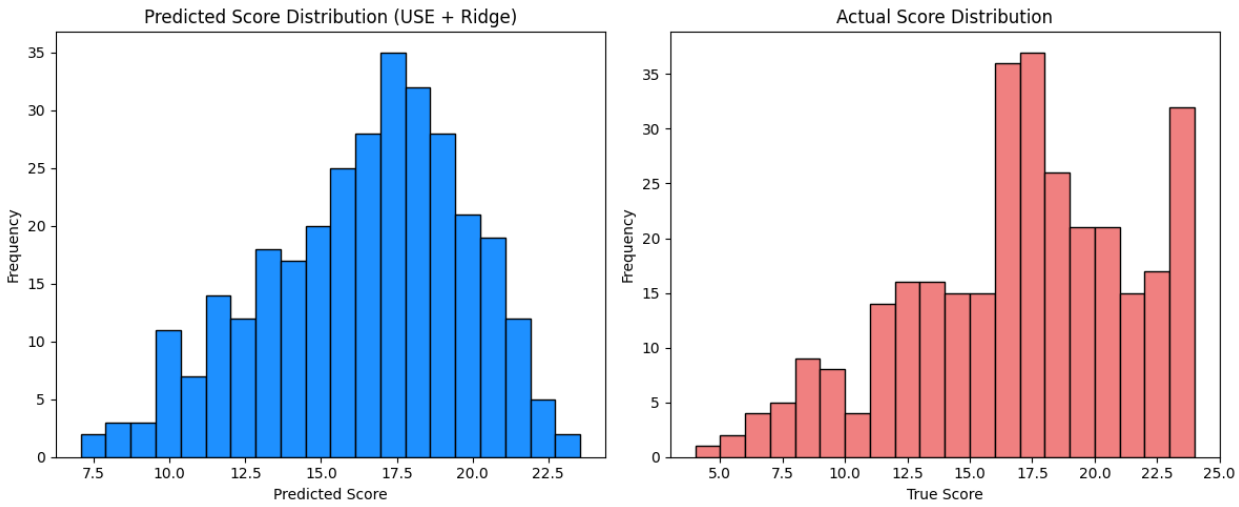
Plot another histogram of the ground truth scores.

How is your new model's distribution of scores different from the ground truth distribution? Is it doing better than your earlier models? How is it doing better?

Graded Cell

This cell is worth 5% of the grade for this assignment.

```
import matplotlib.pyplot as plt

# predict scores using the trained Ridge model
ypred_ridge = ridge_model.predict(xtest)

# plot predicted scores distribution
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(ypred_ridge, bins=20, color='dodgerblue', edgecolor='black')
plt.title("Predicted Score Distribution (USE + Ridge)")
plt.xlabel("Predicted Score")
plt.ylabel("Frequency")

# plot true scores distribution
plt.subplot(1, 2, 2)
plt.hist(ytest, bins=20, color='lightcoral', edgecolor='black')
plt.title("Actual Score Distribution")
plt.xlabel("True Score")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```



After applying the Universal Sentence Encoder (USE) with Ridge regression, the error significantly decreased, and the prediction plot aligned much more closely with the ground truth distribution. The primary reason for this improvement is that USE generates 512-dimensional embeddings, which capture a wide range of linguistic and semantic nuances. These dense, contextual sentence vectors allow the model to better understand the structure, tone, and behavior of an essay. Unlike traditional approaches that rely on sparse, surface-level features

like word counts or TF-IDF, USE Ridge leverages deep sentence embeddings that encode the underlying meaning of the text. As a result, it generalizes more effectively in natural language tasks. In essence, USE Ridge benefits from a much smarter and semantically richer input—it's like feeding the model concepts and intentions rather than just words.

Q11: Plot the errors

We will analyze the difference between the neural model and your best conventional model.

Plot the distribution of errors -- see where the two models made mistakes.

The errors are your model's predicted score minus the ground truth human score.

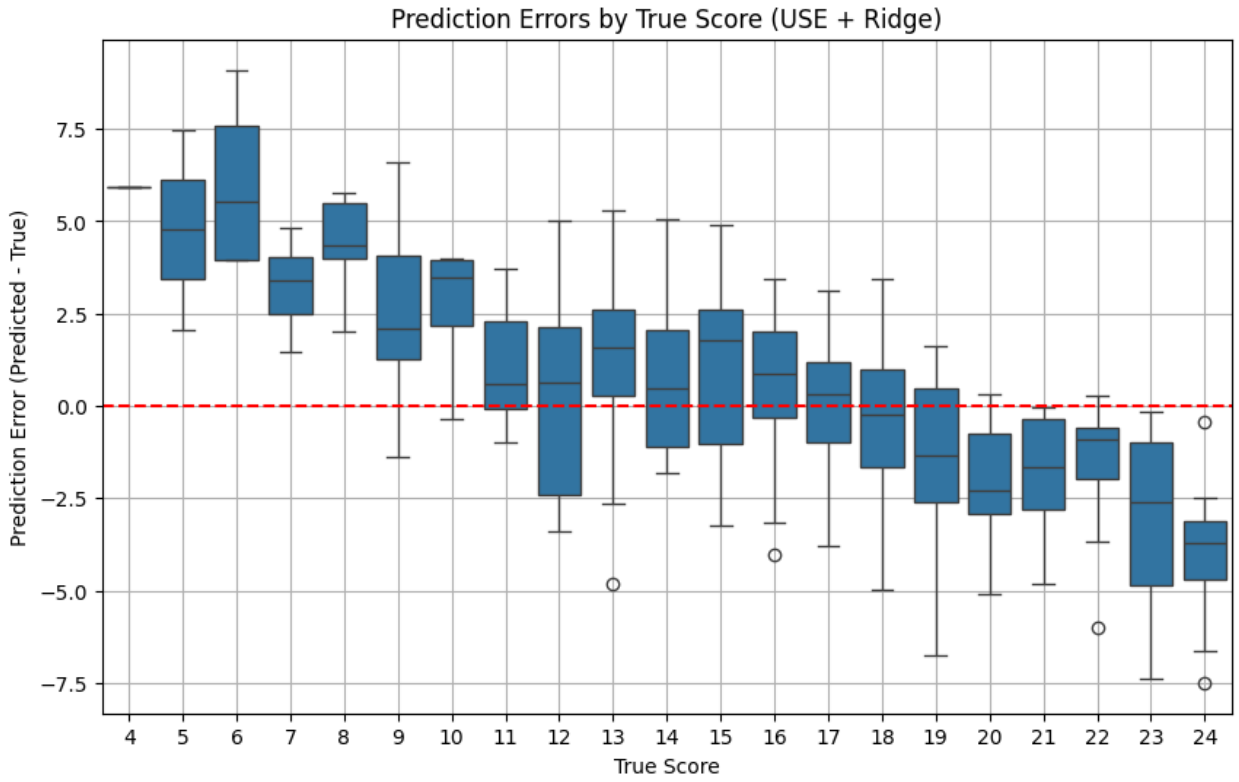
Plot a boxplot of the errors for your model using the universal sentence encoder. Use the seaborn histplot function.

x will be the ground truth scores and y is the difference between ground truth and your predictions.

Graded Cell

This cell is worth 5% of the grade for this assignment.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Compute errors
errors_use = ypred_ridge - ytest

# Create DataFrame
error_df_use = pd.DataFrame({
    'True Score': ytest,
    'Prediction Error': errors_use
})

# Plot: Boxplot of errors by True Score
plt.figure(figsize=(10, 6))
sns.boxplot(x='True Score', y='Prediction Error', data=error_df_use)
plt.axhline(0, color='red', linestyle='--')
plt.title("Prediction Errors by True Score (USE + Ridge)")
plt.xlabel("True Score")
plt.ylabel("Prediction Error (Predicted - True)")
plt.grid(True)
plt.show()
```



Q12: Compare models directly

Plot a histogram of the difference between your neural model and the ground truth.

Plot another histogram of the difference between your best conventional model and the ground truth.

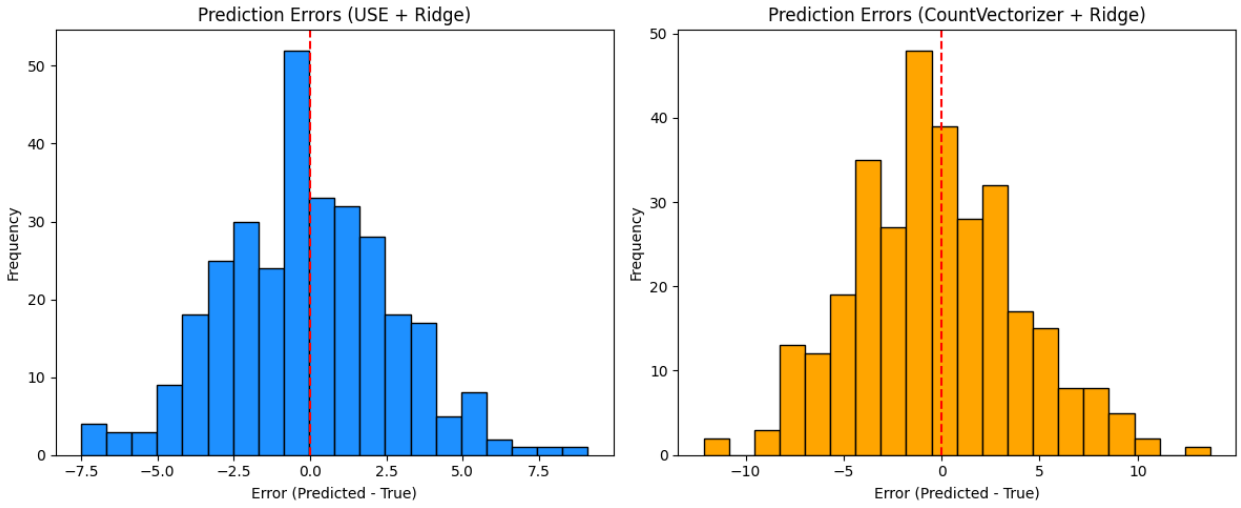
Does either model tend to overestimate or underestimate the true score?

Graded Cell

This cell is worth 10% of the grade for this assignment.

```
# Compute errors
errors_use = ypred_ridge - ytest         # USE Ridge
errors_conv = y_pred - ytest # Conventional Ridge

# Plot histograms
plt.figure(figsize=(12, 5))

# USE Ridge
plt.subplot(1, 2, 1)
plt.hist(errors_use, bins=20, color='dodgerblue', edgecolor='black')
plt.title("Prediction Errors (USE + Ridge)")
plt.xlabel("Error (Predicted - True)")
plt.ylabel("Frequency")
```

```
plt.axvline(0, color='red', linestyle='--')

# Conventional Ridge
plt.subplot(1, 2, 2)
plt.hist(errors_conv, bins=20, color='orange', edgecolor='black')
plt.title("Prediction Errors (CountVectorizer + Ridge)")
plt.xlabel("Error (Predicted - True)")
plt.ylabel("Frequency")
plt.axvline(0, color='red', linestyle='--')

plt.tight_layout()
plt.show()
```



Answer the questions here

Q13: Summarize your findings

Summarize your results. Which approach worked best? Why? Does automatic essay scoring appear feasible? How might we improve on this model?

Graded Cell

This cell is worth 15% of the grade for this assignment.

Write your answers to Q13 here:

Summery: In this assignment, two approaches were compared for automatic essay scoring: a conventional Ridge regression model using CountVectorizer features, and a more advanced model using Universal Sentence Encoder (USE) embeddings with Ridge regression.

Results showed that the USE + Ridge model significantly outperformed the conventional method in both predictive accuracy and error distribution. The USE model produced a higher R^2

score and lower mean squared error, and its error histogram was more tightly centered around zero, indicating more consistent and reliable predictions.

The key reason for this improvement lies in the nature of the input features. While CountVectorizer relies on sparse, surface-level word frequencies, USE provides dense, semantic embeddings that capture the meaning and structure of full sentences. This allows the model to better understand essay content, tone, and coherence—key aspects of writing quality.

These results suggest that automatic essay scoring is feasible, especially when using modern deep language representations like USE. However, there is room for improvement. Future models could integrate attention-based architectures (e.g., BERT or fine-tuned transformers), consider multiple aspects of scoring (e.g., grammar, coherence, relevance), and apply multi-task learning or ensemble methods. Additionally, human-in-the-loop systems could further enhance reliability and fairness in real-world applications.